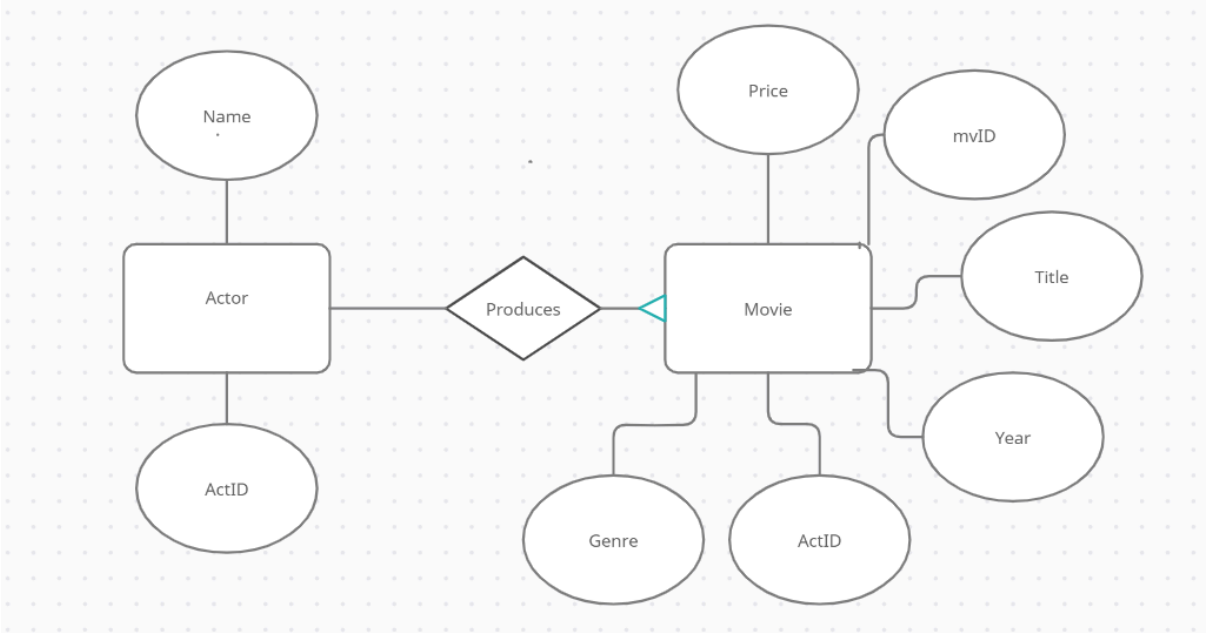
E/R Diagram



The Entity-Relationship diagram portrays the relationship one-to-many, with one actor producing many movies. The relationship, produces, is shown through the diamond. The entities are the rectangle shapes with curved edges, Movie and Actor, with the attributes of each being represented as ovals. We can see foreign key ActID represented in Movie AND Actor rather than just Actor. Every other attribute is local to its entity.

Code

Create Table

Actor:

```
CREATE TABLE Actor ( actID INT NOT NULL AUTO_INCREMENT,
                     actName VARCHAR(255) NOT NULL,
                     CONSTRAINT pk_actor PRIMARY KEY (actID),
                     CONSTRAINT ck_actor UNIQUE (actName)) ENGINE = InnoDB;
```

Movie:

```
CREATE TABLE Movie ( mvID INT NOT NULL AUTO_INCREMENT,
                      actID INT NOT NULL,
                      genre VARCHAR(255) NOT NULL,
                      year INT NULL,
                      title VARCHAR(255) NOT NULL,
                      price REAL NOT NULL,
                      CONSTRAINT pk_mv PRIMARY KEY (mvID),
                      CONSTRAINT fk_mv_act FOREIGN KEY (actID) REFERENCES
                      Actor (actID)
                     ) ENGINE = InnoDB;
```

The AUTO_INCREMENT was added to actID and mvID for the tables Actor and Movie, respectively, so that the first time a row was inserted into Actor, it was given actID number 1, and the first row inserted into Movie was given mvID number 1, then every following row that was inserted into either table would have its IDs increase by 1.

Populate Table

INSERT INTO Actor (actName) VALUES (‘Barry’);
INSERT INTO Actor (actName) VALUES (‘Bullet’);
INSERT INTO Actor (actName) VALUES (‘Jonny’)
INSERT INTO Actor (actName) VALUES (‘Daniel’);
INSERT INTO Actor (actName) VALUES (‘James’);
INSERT INTO Actor (actName) VALUES (‘Arethan’);
INSERT INTO Actor (actName) VALUES (‘Ryan’);
INSERT INTO Actor (actName) VALUES (‘Laura’);
INSERT INTO Actor (actName) VALUES (‘Sebastian’);

The actors were added into the table Actor in this order to match the criteria, with ‘Barry’ being inserted first to have actID 1 and each new insertion would increase actID by 1.

INSERT INTO Movie (actID, title, price, genre) VALUES (6, ‘Die hard with a vengeance’, 12.24, ‘Action’);
INSERT INTO Movie (actID, title, price, genre) VALUES (1, ‘Black Snake Moan’, 9.99, ‘Adventure’);
INSERT INTO Movie (actID, title, price, genre) VALUES (2, ‘Snake on a Plane’, 9.99, ‘Sci-Fi’);
INSERT INTO Movie (actID, title, price, genre) VALUES (4, ‘Freeway of Love’, 8.05, ‘Drama’);
INSERT INTO Movie (actID, title, price, genre) VALUES (5, ‘I knew you were waiting for me’, 12.24, ‘Comedy’);
INSERT INTO Movie (actID, title, price, genre) VALUES (3, ‘Black Panther’, 9.99, ‘Drama’);
INSERT INTO Movie (actID, title, price, genre) VALUES (8, ‘The Jungle Book’, 9.99, ‘Crime’);
INSERT INTO Movie (actID, title, price, genre) VALUES (7, ‘Infinity War’, 7.99, ‘Horror’);
INSERT INTO Movie (actID, title, price, genre) VALUES (9, ‘Coming to Europe’, 8.54, ‘Adventure’);

The movies were added into the table Movie in this order to match the criteria, with ‘Die hard with a vengeance’ being inserted first to have mvID 1 and each new insertion would increase mvID by 1. actID was specified for each insertion as it was a foreign key that joined the two tables Actor and Movie together, and matched each actor with the movie they produced.

COMP1004 Coursework 1

Feedback:

Check	Result	Notes
Database connecttion:	Pass	Successful connection to database 'psynb8_COMP1004' for user 'psynb8_COMP1004'.
Tables description:	Pass	2 tables found in datbase 'psynb8_COMP1004' for user 'psynb8_COMP1004': Actor Movie
Tables rows:	Pass	Table 'Actor' has 9 rows. Table 'Movie' has 9 rows.

Result registration:

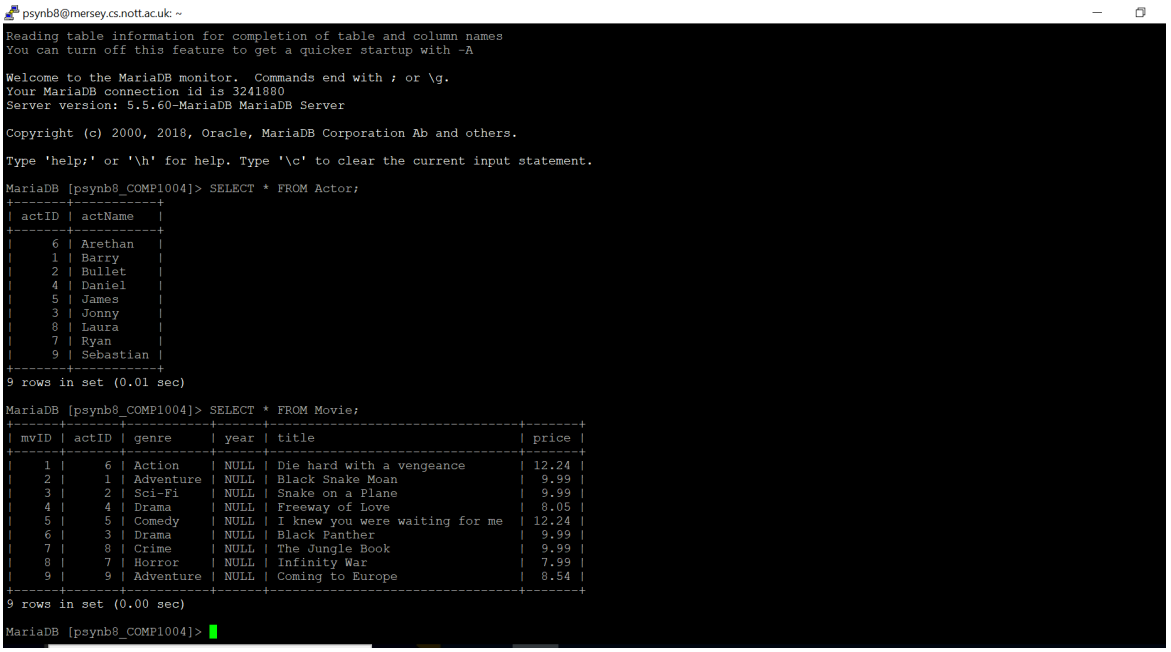
Successfully registered record for student 'psynb8'

Search Contents of Table

SELECT * FROM Movie;

SELECT * FROM Actor;

To show both tables, you simply use the ‘SELECT’ ‘FROM’ keywords along with the wildcard, *, in order to retrieve everything, as well as the table name to know what should be shown.



Update Table

UPDATE Actor SET actName = ‘Barry Nelson’ WHERE actName = ‘Barry’;

UPDATE Actor SET actName = ‘James Bond ’WHERE actName = ‘James’;

The ‘UPDATE’ and ‘SET’ were used to alter values within the table, changing to full names rather than first names for some actors. This was done by specifying ‘WHERE’ the changes had to be, in actName, and what to ‘SET’ the new actName to.

In order to add £1 to the price of each movie, the table Movie had to be updated via the ‘UPDATE’ keyword and have £1 added to its ‘price’ attribute using ‘SET’.

UPDATE Movie SET price = price+1;

To revert to the original price, simply repeat the previous step but rather than using +, you use, -

UPDATE Movie SET price = price-1;

In order to decrease the price of anything worth over £10, the same instructions are followed, however there are 2 changes, one minor and one major. Firstly, the price deducted must be 0.5 instead of 1. Secondly, it must be declared that the price of the movie only changes if its original price is over £10. This is done using the ‘WHERE’ keyword, which specifies it so that the price will remain the same if the movie is less than £10.

UPDATE Movie SET price = price-0.5 WHERE price>10;

Search Updated Table

```
MariaDB [psynb8_COMP1004]> SELECT * FROM Movie;
+------+-------+-----------+------+--------------------------------+-------+
| mvID | actID | genre     | year | title                          | price |
+------+-------+-----------+------+--------------------------------+-------+
|    1 |     6 | Action    | NULL | Die hard with a vengeance      | 11.74 |
|    2 |     1 | Adventure | NULL | Black Snake Moan               |  9.99 |
|    3 |     2 | Sci-Fi    | NULL | Snake on a Plane               |  9.99 |
|    4 |     4 | Drama     | NULL | Freeway of Love                |  8.05 |
|    5 |     5 | Comedy    | NULL | I knew you were waiting for me | 11.74 |
|    6 |     3 | Drama     | NULL | Black Panther                  |  9.99 |
|    7 |     8 | Crime     | NULL | The Jungle Book                |  9.99 |
|    8 |     7 | Horror    | NULL | Infinity War                   |  7.99 |
|    9 |     9 | Adventure | NULL | Coming to Europe               |  8.54 |
+------+-------+-----------+------+--------------------------------+-------+
9 rows in set (0.00 sec)

MariaDB [psynb8_COMP1004]> SELECT * FROM Actor;
+-------+--------------+
| actID | actName      |
+-------+--------------+
|     6 | Arethan      |
|     1 | Barry Nelson |
|     2 | Bullet       |
|     4 | Daniel       |
|     5 | James Bond   |
|     3 | Jonny        |
|     8 | Laura        |
|     7 | Ryan         |
|     9 | Sebastian    |
+-------+--------------+
9 rows in set (0.00 sec)

MariaDB [psynb8_COMP1004]>
```

Delete contents from Table

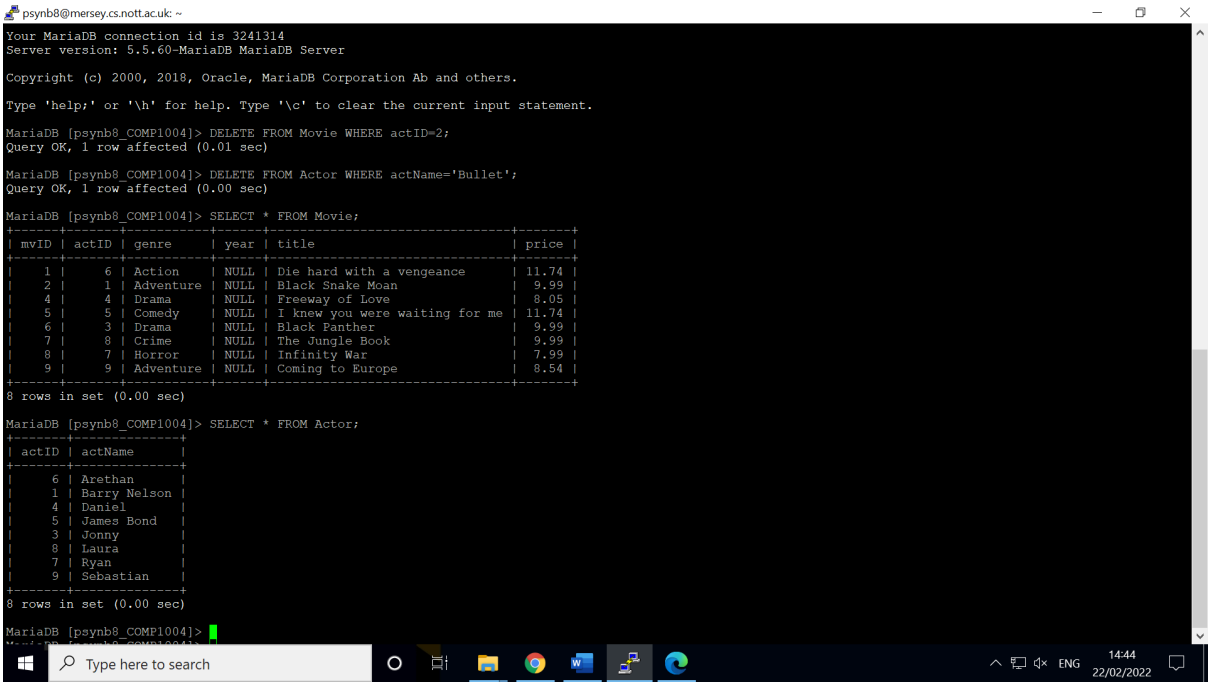
DELETE FROM Actor WHERE actName='Bullet';

The picture below depicts the error the above code produces. This error is a cause of attempting to remove a row from Actor, however cannot be done as it is also a row in Movie due to the join actID creates between the two tables. If we were to delete the actor Bullet, the actID reference in Movie for their film would be invalid. As a result, this row cannot be deleted from Actor unless first removed from Movie.This is known as a foreign key constraint.

```
MariaDB [psynb8_COMP1004]> DELETE FROM Actor WHERE actName='Bullet';
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`psynb8_COMP1004`.`Movie`, CONSTRAINT `fk_mv_act` FOREIGN KEY (`act
ID`) REFERENCES `Actor` (`actID`))
MariaDB [psynb8_COMP1004]>
```

FIXED VERSION

The picture below shows the table with one removed row, ‘WHERE’ actName was ‘Bullet’. To remove the row, firstly the actID of Bullet, 2, had to be removed from Movie so there was no join between Actor and Movie for Bullet. Now that the foreign key actID was not linking ‘Bullet’ Actor to the Movie table, we can simply re-run the earlier code again and have ‘Bullet’ removed with no errors.



COMP1004 Coursework 1

Feedback:

Check	Result	Notes
Database connecttion:	Pass	Successful connection to database 'psynb8_COMP1004' for user 'psynb8_COMP1004'.
Tables description:	Pass	2 tables found in datbase 'psynb8_COMP1004' for user 'psynb8_COMP1004': Actor Movie
Tables rows:	Pass	Table 'Actor' has 8 rows. Table 'Movie' has 8 rows.

Result registration:

Successfully registered record for student 'psynb8'